AZ-400: Designing and Implementing Microsoft DevOps Solutions

Complete Study Guide (Merged Version)

Last Updated: December 2024 **Exam:** Designing and Implementing Microsoft DevOps Solutions **Prerequisites:** Must hold AZ-104 (Azure Administrator) OR AZ-204 (Azure Developer) certification **Duration:** 140-180 minutes **Passing Score:** 700/1000 **Certification:** Microsoft Certified: Azure DevOps Engineer Expert

---

EXAM WEIGHTAGE BY SECTION

Section	Weight	Key Focus Areas
Design and implement build and release pipelines	50-55%	CI/CD, YAML, deployment strategies, testing, infrastructure as code, package management
Design and implement processes and communications	10-15%	Traceability, metrics dashboards, collaboration tools, feedback cycles
Design and implement a source control strategy	10-15%	Git branching, repository management, GitHub/ Azure Repos, authentication
Develop a security and compliance plan	10-15%	Authentication, secrets management, security scanning, compliance automation
Implement an instrumentation strategy	5-10%	Azure Monitor, Application Insights, KQL, monitoring configuration

---

OFFICIAL MICROSOFT RESOURCES

- **Study Guide**: https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/az-400
- **Exam Readiness Zone**: https://learn.microsoft.com/en-us/shows/exam-readiness-zone/preparing-for-az-400-design-and-implement-processes-and-communications
- **Training Path**: https://learn.microsoft.com/en-us/training/paths/az-400-manage-source-control/
- **GitHub Integration**: https://learn.microsoft.com/en-us/azure/devops/cross-service/github-integration
- **YAML Schema Reference**: https://learn.microsoft.com/en-us/azure/devops/pipelines/yaml-schema/
- **GitHub Actions**: https://docs.github.com/en/actions
- **Service Principals**: https://learn.microsoft.com/en-us/azure/devops/integrate/get-started/authentication/service-principal-managed-identity

- **Key Vault Integration**: https://learn.microsoft.com/en-us/azure/devops/pipelines/release/azure-key-vault
- **GitHub Advanced Security**: https://docs.github.com/en/github/administering-a-repository/about-github-advanced-security
- **Azure Monitor**: https://learn.microsoft.com/en-us/azure/azure-monitor/overview
- **Application Insights**: https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview
- **KQL Training**: https://learn.microsoft.com/en-us/azure/data-explorer/kusto/query/

---

1. DESIGN AND IMPLEMENT PROCESSES AND COMMUNICATIONS (10-15%)

1.1 Design and Implement Traceability and Flow of Work

GitHub Flow Implementation

- **Lightweight, branch-based workflow**: Main branch always deployable, feature branches for all changes
- **Pull requests**: Mandatory for code review, merge after approval, immediate deployment
- **Key Metrics**: Cycle time, lead time, time to recovery (MTTR), deployment frequency
- **Documentation**: https://docs.github.com/en/get-started/quickstart/github-flow
- **Branch Protection Rules**: Require status checks, enforce code reviews, restrict who can push to main

Feedback Cycles

- **GitHub Issues**: Create issue templates, label management, project automation, close issues via commit messages
- **Azure Boards Integration**
  :
  - Work item types: Epics, Features, User Stories, Tasks, Bugs
  - Custom fields and process customization (Agile, Scrum, CMMI)
  - Link Azure Boards work items to GitHub commits/PRs using AB#123 syntax
  - Connect repositories to Azure Boards for work item tracking
  - Use #ID syntax in commit messages to link commits to work items
  - Configure automatic status updates based on pull request states
  - Implement custom workflows using process rules

- **Notifications**: Configure GitHub notifications (Watch, Subscriptions), Azure DevOps notifications (Email, Teams, Slack), webhook integrations
- **Sprint reviews and retrospectives** for process feedback

Source Traceability

- **GitHub**: Trace commits to PRs to issues to releases, use GitHub Projects for tracking, enable commit signing verification
- **Azure Repos**: Branch policies, path policies, require linked work items, enforce comment resolution
- **Cross-platform**: Use Azure DevOps service connections to GitHub, configure bidirectional linking
- **Achieve end-to-end traceability** linking commits, work items, and releases

1.2 Design and Implement Metrics and Queries

Azure Boards Dashboards

- **Widgets**: Lead Time & Cycle Time, Burndown/Burnup charts, Velocity, Query Tiles, Count Query, Cumulative Flow Diagram (CFD), Test Results Trend, Build History
- **Custom Queries**: Work Item Query Language (WIQL), @CurrentIteration, @Me, @Project macros
- **Analytics Views**: Enable Analytics service, create shared views, use OData for advanced reporting
- **Configure query-based charts and widgets**

GitHub Projects

- **Project Views**: Board, Table, Roadmap layouts, custom filters, group by sprint/milestone
- **Insights**: Burndown charts, velocity tracking, cumulative flow diagrams
- **Metrics**: PR merge time, issue resolution time, contributor activity, code review turnaround

DevOps Metrics by Phase

- **Planning**: Sprint velocity, backlog health, story point completion rate, sprint burndown, capacity metrics
- **Development**: PR size, commit frequency, build success rate, code churn, pull request metrics
- **Testing**: Test pass rate, code coverage, defect density, test execution time
- **Security**: Vulnerability scan results, dependency audit failures, secret detection count, vulnerability counts
- **Delivery**: Deployment frequency, change failure rate, mean time to restore (MTTR)
- **Operations**: Application availability, performance baselines, alert noise ratio

KQL for Azure DevOps

```
WorkItemBoardSnapshot
| where TeamName == "YourTeam"
| summarize Count=count() by WorkItemType, ColumnName
| render piechart
```

Feedback Cycle Strategy

- Use automated notifications via webhooks and service hooks
- Create dashboards showing cycle times, lead times, and time to recovery
- **Key Metrics Definitions**
  :
  - **Cycle Time**: Time from work start to completion
  - **Lead Time**: Time from request to delivery
  - **Time to Recovery**: Time from failure to service restoration
  - **Failure Rate**: Percentage of deployments causing failures

1.3 Configure Collaboration and Communication

Wiki Documentation

- **Azure DevOps Wiki**: Markdown syntax, Mermaid diagrams, embed work item queries, link code, version control integration
- **GitHub Wiki**: Separate repo, Markdown support, sidebar navigation, link to main repo files
- **Documentation as Code**: Store docs in /docs folder, automate publishing via pipelines
- **Create project wikis** using Markdown syntax, embed Mermaid diagrams for process visualization
- **Automate documentation generation** from Git history

Release Documentation

- **Azure DevOps**: Auto-generate release notes from work items, use Release Notes Generator task
- **GitHub**: Auto-generate from PR titles, use GitHub Releases API, customize with release.yml configuration
- **API Documentation**: Swagger/OpenAPI integration, publish to Azure API Management, automate with Swashbuckle
- **Generate release notes automatically** from linked work items
- **Publish API documentation** using Swagger/OpenAPI
- **Document deployment processes** in wiki pages

Webhook Configuration

- **Azure DevOps Service Hooks**: JSON payload, filter by event type, authentication options (basic, token)
- **GitHub Webhooks**: Secret token validation, SSL verification, event types (push, PR, release)
- **Integration Targets**: Microsoft Teams, Slack, custom HTTP endpoints, Azure Functions
- **Documentation**: https://learn.microsoft.com/en-us/azure/devops/service-hooks/overview
- **Subscribe to events**: code push, pull request, work item updates
- **Configure payload URL and secret tokens**
- **Filter events by event type and resource type**

Teams Integration

- **Azure Boards**: Add Boards tab in Teams, create work items from chat, @mention work items
- **GitHub**: GitHub app for Teams, PR notifications, issue updates, code push alerts
- **Azure Pipelines**: Subscribe to pipeline notifications, approve releases from Teams
- **Microsoft Teams Integration**: Use Azure DevOps bot for notifications

---

2. DESIGN AND IMPLEMENT A SOURCE CONTROL STRATEGY (10-15%)

2.1 Design and Implement Branching Strategies

GitHub Flow (Simple Projects)

- Main branch always deployable
- Feature branches: feature/description or username/feature
- Pull requests mandatory for review
- Merge squash or rebase for clean history
- **Documentation**: https://docs.github.com/en/get-started/quickstart/github-flow

Git Flow (Complex Releases)

- Main, develop, feature/*, release/*, hotfix/* branches
- Version tagging on main branch
- Long-running release branches for staging
- **Tooling**: Use git-flow CLI or GitHub Desktop integrations

Trunk-Based Development (High Velocity)

- Short-lived feature branches (< 1 day)
- Feature flags for incomplete features
- Continuous integration on every commit
- Pair programming for real-time review

Feature Branch Workflow

- Create feature branches from main: feature/feature-name or users/username/description
- Use bugfix branches: bugfix/description
- Implement hotfix branches for critical production issues
- Never commit directly to main branch

Release Branch Management

- Create release branches from main: release/2.0
- Implement cherry-pick for porting fixes back to main
- Lock release branches when support ends
- Use tags for release versioning: v1.0.0, v2.1.0

Branch Protection Configuration

GitHub branch protection rules:

```
branches:
  - name: main
    protection:
      required_status_checks:
        strict: true
        contexts: [ci/build, ci/test]
      required_pull_request_reviews:
        required_approving_review_count: 2
        require_code_owner_reviews: true
      enforce_admins: true
      required_linear_history: true
      restrictions: null
```

Azure Repos Branch Policies:

- Require minimum number of reviewers (typically 2)
- Require linked work items
- Enforce comment resolution
- Build validation pipeline
- Status checks from external services
- Merge restrictions:
  - Disable "Allow requestors to approve their own changes"
  - Enable "Reset all approval votes when new changes are pushed"

2.2 Configure and Manage Repositories

GitHub Enterprise Management

- **Organizations**: Member roles (Owner, Member, Billing manager), teams with nested hierarchy, repository roles (Admin, Write, Read, Triage)
- **Repository Settings**: Disable forking, restrict pushes, enable vulnerability alerts, secret scanning, push protection
- **Large File Management**: Git LFS configuration, migrate large files, storage quotas (for binaries >100MB)
- **Repository Scalability**: Use Git VFS (Scalar) for monorepo optimization, sparse-checkout,

shallow clones
- **Connect organization to Microsoft Entra ID** for identity management
- **Enable two-factor authentication** for sensitive projects
- **Configure permission levels**: Read, Contribute, Admin
- **Use teams for permission management**

Azure Repos Configuration

- **Repository Structure**: Separate repos by bounded context, use Git submodules for shared libraries
- **Permissions**: Grant at project/repo level, use groups for scale, restrict on branches/tags
- **Pull Request**: Require build validation, enforce policy compliance, configure auto-complete
- **Fork Workflow**: Enable fork syncing, PR across forks, permissions for external contributors
- **Repository Settings**
  :
  - Enable push protection for secrets
  - Configure retention policies for deleted branches
  - Set maximum file size limits
  - Enable automatic linking of work items

Cross-Repository Sharing

- **GitHub**: Use internal repositories for InnerSource, repository templates, GitHub Packages for shared components
- **Azure DevOps**: Use Git submodules, Azure Artifacts for package sharing, service connections for authentication

Code Review Automation

- **GitHub**: CODEOWNERS file, required status checks, auto-assign reviewers, PR templates
- **Azure DevOps**: Required reviewers, policy enforcement, comment resolution, build gates

Authentication Strategies

- **Work identity federation**: Use managed identities or app registrations
- **Personal Access Tokens (PATs)**: Scoped tokens with expiration dates
- **SSH Keys**: For command-line Git operations

Scaling Git Repositories

- Use Git LFS (Large File Storage) for binaries >100MB
- Implement Scalar for monorepo optimization
- Enable cross-repository sharing for common components

---

3. DESIGN AND IMPLEMENT BUILD AND RELEASE PIPELINES (50-55%)

3.1 Design and Implement Package Management Strategy

Tool Selection

- **Azure Artifacts**: NuGet, npm, Python, Maven, Universal packages; upstream sources to public registries (npmjs, PyPI); feed views (@local, @prerelease, @release)
- **GitHub Packages**: Container registry, npm, NuGet, Maven, RubyGems; seamless integration with GitHub Actions; fine-grained permissions via repository access
- **Decision Factors**: Team size, external collaboration needs, existing ecosystem, cost considerations

Feed Configuration

Azure Pipelines - Publish to Azure Artifacts:

```
- task: NuGetCommand@2
  inputs:
    command: 'push'
    packagesToPush: '${Build.ArtifactStagingDirectory}/*.nupkg'
    nuGetFeedType: 'internal'
    publishVstsFeed: '/'
```

GitHub Actions - Publish to GitHub Packages:

```
- name: Publish to GitHub Packages
  run: dotnet nuget push *.nupkg --source "github"
  env:
    NUGET_AUTH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Azure Artifacts Configuration

- Create feeds for different package types: NuGet, npm, Maven, Python
- Upstream sources: Enable upstream sources to public registries
- Retention policies: Set package retention limits
- Permissions: Configure feed permissions: Owner, Contributor, Reader

Package Promotion

- Use views to promote packages: @Local, @Prerelease, @Release

- Automate package promotion via pipelines
- Implement package immutability for released versions

Package Consumption

- Configure .npmrc, nuget.config, pom.xml to use Azure Artifacts
- Use upstream sources to cache external packages
- Implement package vulnerability scanning

Versioning Strategy

- **Semantic Versioning (SemVer)**: MAJOR.MINOR.PATCH format, automated versioning through GitVersion or Nerdbank.GitVersioning
- **Calendar Versioning (CalVer)**: YYYY.0M.MICRO format for time-based releases
- **Pipeline Artifacts**: Version with build ID, source version, and branch name; use publishBuildArtifacts task with semantic paths
- **Use semantic versioning for releases**

Dependency Management

- **Dependabot**: Configuration file .github/dependabot.yml, daily/weekly scans, auto-merge rules, security updates only
- **Azure DevOps**: Enable Automated Security Updates, configure package rules, global approval policies
- **Lock Files**: Commit package-lock.json, nuget.lock.json, Pipfile.lock to ensure reproducible builds

3.2 Design and Implement Testing Strategy for Pipelines

Test Types and Implementation

- **Unit Tests**: Run on every PR, parallel execution, code coverage thresholds (minimum 80%)
- **Integration Tests**: Run post-merge, require live services, use testcontainers or Docker Compose
- **Functional Tests**: Selenium/Playwright for UI testing, API testing with Postman/Newman
- **Load Tests**: Azure Load Testing service, JMeter integration, run on schedule or release gate
- **Security Tests**: Static analysis (SonarQube, CodeQL), dependency scanning (Snyk, OWASP Dependency-Check), secrets detection (GitHub Secret Scanning, Microsoft Defender)

Pipeline Test Configuration

Azure Pipelines - Multi-stage testing:

```
stages:
  - stage: Unit_Tests
    jobs:
      - job: Test
        steps:
          - task: DotNetCoreCLI@2
            inputs:
              command: 'test'
              projects: '*/*Tests.csproj'
              arguments: '-collect:"XPlat Code Coverage"'

  - stage: Integration_Tests
    dependsOn: Unit_Tests
    condition: succeeded()
    jobs:
      - job: Test
        steps:
          - task: AzureWebApp@1
            inputs: # Deploy to test environment
          - script: npm run test:integration
```

Build Tasks

- Code compilation: Use appropriate build tasks (MSBuild, Maven, npm)
- Unit testing: Run tests with code coverage collection
- Static code analysis: Integrate SonarQube, CodeQL
- Security scanning: Run dependency scanning, secret scanning
- Artifact publishing: Publish build artifacts using AzurePipelineArtifacts@1

Quality Gates

- **Azure DevOps**: Gates and approvals on environments, invoke REST API checks, query work items, evaluate Azure Monitor alerts
- **GitHub**: Environment protection rules, required status checks, branch protection rules
- **Criteria**: Minimum code coverage, zero critical vulnerabilities, performance benchmarks met, security scan pass

Code Coverage Analysis

- **Tools**: Coverlet, JaCoCo, Istanbul, Cobertura format for universal reporting
- **Publish**: Use PublishCodeCoverageResults@1 task, integrate with SonarQube, enforce coverage gates
- **Exclusions**: Configure .runsettings or coverage.yml to exclude generated code, tests, and third-party libraries

Build Optimization

- **Caching**: Cache dependencies (NuGet, npm, Maven)
- **Parallel jobs**: Run tests in parallel across multiple agents
- **Matrix strategy**: Test across multiple configurations

- **Conditional execution**: Use condition for selective task execution
- **Incremental builds**: Use clean: false for faster builds
- **Parallel compilation**: Enable /m flag for MSBuild
- **Test parallelization**: Configure parallel: true in test tasks

Build Triggers

- **Continuous Integration**: Trigger on every commit
- **Scheduled builds**: Run nightly builds
- **Pull request validation**: Trigger on PR creation/update
- **Path filters**: Trigger only when specific paths change

3.3 Design and Implement Pipelines

YAML Pipeline Structure

```
trigger:
  branches:
    include:
      - main
    exclude:
      - feature/*
pool:
  vmImage: 'ubuntu-latest'

stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - checkout: self
          - task: DotNetCoreCLI@2
            inputs:
              command: 'build'
              projects: '**/*.csproj'
```

Agent Infrastructure

- **Microsoft-Hosted Agents**: Pre-configured VMs (Ubuntu, Windows, macOS), automatic updates, 360 minutes timeout, 10 GB storage
- **Self-Hosted Agents**: Install on Azure VMs, on-premises servers, or containers; benefits: custom software, persistent workspace, cost savings for high volume
- **GitHub Runners**: GitHub-hosted (2-core, 7 GB RAM) or self-hosted runners; use runner groups for organization-level management
- **Container Jobs**: Run pipeline steps inside containers, ensure consistency, isolate dependencies
- **Scale sets**: Use Azure Virtual Machine Scale Sets for auto-scaling
- **Agent pools**: Organize agents by capability or environment

Agent Pool Configuration

Azure Pipelines:

```
pool:
  name: 'MyAgentPool'
  demands:
    - java -equals 17
    - Agent.OS -equals Linux
```

GitHub Actions:

```
runs-on: [self-hosted, Linux, X64]
container: mcr.microsoft.com/dotnet/sdk:6.0
```

Trigger Rules

- **CI Triggers**: Branch filters (include: [main, develop]), path filters (exclude: ['docs/*']), batch changes, PR triggers
- **Schedule Triggers**: Cron syntax, multiple schedules, only scheduled runs
- **Pipeline Triggers**: Trigger from another pipeline completion, resource triggers for container images
- **Manual Triggers**: Use workflow_dispatch for GitHub, pipeline: resource for Azure DevOps

Multi-Stage Pipeline with Templates

```
trigger:
  - main

stages:
  - stage: Build
    jobs:
      - template: build.yml@templates
        parameters:
          buildConfiguration: 'Release'

  - stage: Deploy_Dev
    dependsOn: Build
    jobs:
      - deployment: Deploy
        environment: 'Dev'
        strategy:
          runOnce:
            deploy:
              steps:
                - template: deploy.yml@templates
                  parameters:
                    environment: 'dev'
```

Reusable Elements

- **YAML Templates**: Parameterized templates for jobs/steps, stored in separate repository, version with tags/branches
- **Variable Groups**: Azure DevOps Library groups, link to Azure Key Vault secrets, scope to environment
- **Task Groups**: Azure DevOps UI-based task composition, parameterize inputs, version control
- **GitHub Reusable Workflows**: Reference workflows from other repositories, use workflow_call event
- **Create reusable templates** for common tasks
- **Parameterize templates** for flexibility
- **Store templates in separate repository**
- **Use template expressions** for conditional logic

Job Execution Strategy

- **Parallelism**: Matrix strategy for multi-platform builds, maxParallel control, fan-out/fan-in patterns
- **Dependencies**: dependsOn for job ordering, condition: succeeded() or failed() or always()
- **Multi-Stage**: Sequential stages, environment-specific approvals, deployment vs build stages

Pipeline Security

- **Environments**: Azure DevOps environments with approvers, checks (Azure Policy, Invoke REST API), deployment history
- **GitHub Environments**: Protection rules (required reviewers, wait timer), environment secrets, deployment branches
- **Approvals**: Manual intervention tasks, pre-deployment gates, business hours restrictions

Pipeline Templates

- Create reusable templates for common tasks
- Parameterize templates for flexibility
- Store templates in separate repository
- Use template expressions for conditional logic

Pipeline Optimization

- Caching: Cache dependencies (NuGet, npm, Maven)
- Parallel jobs: Run tests in parallel across multiple agents
- Matrix strategy: Test across multiple configurations
- Conditional execution: Use condition for selective task execution

3.4 Design and Implement Deployments

Deployment Strategies

Multi-Stage Deployment

```
stages:
  - stage: Dev
    jobs:
      - deployment: DeployDev
        environment: 'Dev'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1

  - stage: Production
    dependsOn: Dev
    jobs:
      - deployment: DeployProd
        environment: 'Production'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1
```

Specific Strategies:

- **Blue-Green**: Two identical environments, instant switch via DNS or load balancer, zero downtime, risk: cost of double resources. Use separate staging/production deployments, deploy to staging environment first, run smoke tests, swap production traffic to staging.
- **Canary**: Deploy to small subset (5%), gradually increase (5% → 25% → 50% → 100%), automated health checks, instant rollback. Use feature flags or traffic routing, gradually increase rollout percentage, monitor metrics before full deployment.
- **Ring-Based**: Deploy to concentric rings (Team → Organization → Partners → Public), based on risk tolerance and user segments.
- **Progressive Exposure**: Feature flags with Azure App Configuration, target users by percentage/group, gradual enablement.
- **A/B Testing**: Split traffic by user attributes, measure business metrics, statistical significance analysis.
- **Rolling deployment**: Deploy to VMs in batches.

```
strategy:
  rolling:
    maxParallel: 2
    preDeploy:
      steps:
        - script: echo "Pre-deployment"
    deploy:
      steps:
        - script: echo "Deploying"
```

Minimize Downtime

- **VIP Swap**: Azure App Service deployment slots, warm-up NewRelic/ping tests, instant swap

with zero downtime
- **Rolling Deployments**: Kubernetes rolling update strategy, maxUnavailable: 25%, health check endpoints
- **Load Balancer Draining**: Azure Load Balancer connection draining, health probe intervals, backend pool updates
- **Database Deployment**: Use DbUp or Flyway for migrations, run migrations pre-deployment, ensure backward compatibility

Hotfix Path Planning

- **Direct to Production**: Bypass normal pipeline for critical fixes, use cherry-pick to main branch, ensure post-deployment validation
- **Feature Flags**: Disable problematic features instantly without redeployment, use Azure Feature Manager
- **Rollback Strategy**: Automated rollback on health check failures, maintain previous deployment artifacts, database rollback procedures

Resiliency Strategy

- **Health Probes**: Configure liveness/readiness probes for containers, custom health check endpoints
- **Retry Policies**: Implement Polly in applications, exponential backoff, circuit breaker pattern
- **Deployment Slots**: Pre-warm slots, run smoke tests, swap with production, auto-swap for staging

Feature Flags Implementation

```
// Azure App Configuration Feature Manager
if (await featureManager.IsEnabledAsync("NewPaymentGateway"))
{
    // Execute new code path
}
```

- **Configuration**: Use Azure App Configuration with labels (Dev, Prod), filters (Percentage, Targeting, TimeWindow)
- **Management**: Azure portal UI, Azure CLI, automated rollout via pipelines

Database Deployments

- **State-Based**: Use SSDT (SQL Server Data Tools), dacpac deployments, drift detection
- **Migration-Based**: Entity Framework migrations, DbUp scripts, version-controlled SQL scripts
- **Pipeline Integration**: Run migrations as pipeline step, use Azure SQL Database deployment task, enforce transaction rollback on failure

Environment Configuration

- Define environments in Azure DevOps: Dev, QA, Staging, Production
- Configure approval gates and checks

- Set up service connections for each environment
- Implement variable groups for environment-specific values

3.5 Design and Implement Infrastructure as Code (IaC)

IaC Strategy Definition

- **State Management**: Use Terraform remote backend (Azure Storage), Azure Resource Manager template specs, Bicep modules with versioning
- **Source Control**: Store IaC in dedicated repository, enforce PR reviews for infrastructure changes, implement GitOps workflows
- **Testing**: Use terratest for Terraform, ARM template validation (Test-AzResourceGroupDeployment), Bicep linter
- **Deployment Automation**: Azure Pipelines with Azure CLI tasks, GitHub Actions with Azure/login, automated rollback on failure

Configuration Management

- **Azure Automation State Configuration**: PowerShell DSC, compile configurations, assign to nodes, drift reporting
- **Azure Automanage Machine Configuration**: Built-in profiles (Azure Best Practices), custom configurations, automatic remediation
- **Bicep Modules**: Reusable module registry, semantic versioning, parameter files per environment
- **Terraform Workspaces**: Separate environments (dev/staging/prod), shared modules from Terraform Registry

Azure Deployment Environments

- **On-Demand Environments**: Self-service via Azure DevTest Labs, ARM/Bicep templates, cost controls (auto-shutdown, quotas)
- **Ephemeral Environments**: Spin up per PR, run integration tests, tear down after merge
- **Template Catalog**: Curated environment definitions, RBAC controls, approval workflows

Bicep Implementation

```
param location string = resourceGroup().location
param storageAccountName string

resource storageAccount 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: storageAccountName
  location: location
  sku: {
    name: 'Standard_LRS'
  }
```

```
  kind: 'StorageV2'
}
```

IaC Best Practices

- **Modular design**: Create reusable Bicep modules
- **Parameterization**: Use parameters for environment-specific values
- **What-If analysis**: Validate changes before deployment
- **State management**: Use Azure Resource Manager for state (no separate state file)
- **Linting**: Use bicep build and bicep lint for validation

Terraform Integration

- Store state in Azure Storage with encryption
- Use Terraform Cloud for remote execution
- Implement workspace per environment
- Use Azure AD authentication for Terraform

Pipeline Integration

Deploy infrastructure via Bicep:

```
- task: AzureResourceManagerTemplateDeployment@3
  inputs:
    deploymentScope: 'Resource Group'
    azureResourceManagerConnection: 'AzureServiceConnection'
    subscriptionId: '$(subscriptionId)'
    action: 'Create Or Update Resource Group'
    resourceGroupName: '$(resourceGroupName)'
    location: '$(location)'
    templateLocation: 'Linked artifact'
    csmFile: '$(Pipeline.Workspace)/Infrastructure/main.bicep'
    overrideParameters: '-environment $(environment) -resourceBaseName
$(resourceBaseName)'
```

- Use AzureResourceManagerTemplateDeployment@3 task for ARM/Bicep
- Use TerraformTaskV4@4 task for Terraform
- Implement approval gates for production IaC changes
- Run what-if analysis in PR validation

3.6 Maintain Pipelines

Pipeline Health Monitoring

- Monitor pipeline duration trends
- Track failure rates and flaky tests
- Set up alerts for pipeline failures
- Analyze agent capacity and queue times

Pipeline Optimization

- Identify and remove redundant tasks
- Implement test impact analysis
- Use pipeline caching effectively
- Optimize Docker layer caching

Agent Management

- **Microsoft-hosted agents**: Use for standard builds
- **Self-hosted agents**: Use for specialized requirements
- Scale sets: Use Azure Virtual Machine Scale Sets for auto-scaling
- **Agent pools**: Organize agents by capability or environment

Artifact Retention

- Configure build retention policies: keep last 10 builds
- Set release retention: keep releases for 30 days
- Implement manual retention for important builds
- Clean up old artifacts to save storage

---

4. DEVELOP A SECURITY AND COMPLIANCE PLAN (10-15%)

4.1 Design and Implement Authentication and Authorization

Service Principals vs Managed Identities

Feature	Service Principal	System-Assigned Managed Identity	User-Assigned Managed Identity
Use Case	CI/CD pipelines, cross-cloud	Single Azure resource	Multiple Azure resources
Credentials	Manual certificate/ secret	Azure-managed, automatic	Azure-managed, independent
Lifecycle	Manual create/delete	Tied to resource lifecycle	Independent lifecycle
Best For	External apps, hybrid cloud	Azure Functions, App Service	Shared identity scenarios

Implementation Steps

1. Create Service Principal:

```
az ad sp create-for-rbac --name "AzureDevOpsSP" --role contributor --scopes /subscriptions/(subId)
```

- Use certificates (recommended) over client secrets
- Store credentials in Azure Key Vault
- Assign least-privilege RBAC roles

2. Create Managed Identity:

```
# System-assigned
az functionapp identity assign --name MyApp --resource-group MyRG

# User-assigned
az identity create --name MyIdentity --resource-group MyRG
```

- Grant access via Azure portal Identity blade
- Assign Azure RBAC roles and Azure DevOps permissions

GitHub Authentication

- **GitHub Apps**: Installable apps with granular permissions, act on behalf of organization, use JWT for authentication
- **Personal Access Tokens (PATs)**: Classic (repo, workflow, admin:org scopes) vs Fine-grained (repository-specific, expiration, IP allowlist)
- **GITHUB_TOKEN**: Automatically generated, limited to repository workflow, expires after job completion

Azure DevOps Service Connections

- **Azure Resource Manager**: Service principal, managed identity, Azure Classic service management
- **GitHub**: PAT or OAuth, install GitHub App
- **Generic**: Username/password, token-based auth for external services
- **Security**: Use managed identity where possible, rotate PATs regularly, restrict service connection usage to specific pipelines

Azure DevOps Security

- Connect organization to Microsoft Entra ID
- Enable conditional access policies
- Implement MFA for all users
- Use managed identities for service connections

Service Connection Security

```
- task: AzureCLI@2
    inputs:
      azureSubscription: 'MyServiceConnection'
      scriptType: 'bash'
      scriptLocation: 'inlineScript'
```

Service Connection Roles

- Reader: View service connections
- User: Use in pipelines
- Creator: Create new connections
- Administrator: Manage permissions

4.2 Manage Sensitive Information in Automation

Azure Key Vault Integration

Azure Pipelines - Retrieve secrets:

```
- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'AzureServiceConnection'
    KeyVaultName: 'my-keyvault'
    SecretsFilter: '*'
    RunAsPreJob: false
```

GitHub Actions - Azure Key Vault:

```
- uses: azure/login@v1
  with:
    creds: ${{ secrets.AZURE_CREDENTIALS }}
- uses: Azure/get-keyvault-secrets@v1
  with:
    keyvault: "my-keyvault"
    secrets: 'DBPassword,APIToken'
  id: myKeyVault
```

Azure Key Vault Integration Details

- Store secrets, keys, certificates in Key Vault
- Enable Key Vault logging and monitoring
- Configure network restrictions for Key Vault
- Use managed identities for authentication

GitHub Secrets

- **Repository Secrets**: Encrypted at rest, available to workflows, scoped to repository
- **Environment Secrets**: Scoped to specific environment (dev/staging/prod), require approval
- **Organization Secrets**: Available across repos, scoped to selected repositories
- **Secret Scanning**: Enable push protection, receive alerts for leaked secrets, revoke and rotate automatically
- **Use GitHub encrypted secrets** in workflows
- **Environment-level secrets** for production
- **Secret scanning** to prevent leakage

Azure Pipelines Secure Files

- Store certificates, SSH keys, provisioning profiles
- Encrypted storage, downloaded during pipeline execution
- Variable substitution for connection strings, config files
- Access permissions at project level

Pipeline Design for Secret Prevention

- Never log secrets: Use setSecret logging command to mask values
- Override config at runtime: Use environment variables, configuration transforms
- Audit pipeline runs: Review logs for accidental exposure, enable log retention policies

Pipeline Secrets

- Mark variables as secret: isSecret: true
- Use secret variables in tasks:

```
- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'MySubscription'
    KeyVaultName: 'my-keyvault'
    SecretsFilter: '*'
```

Secret Management

- Use Azure Key Vault for secrets
- Reference secrets in pipelines:

```
variables:
  mySecret: ${mySecretFromKeyVault}
```

Credential Scanning

- Enable GitHub Advanced Security secret scanning
- Use CredScan in Azure Pipelines
- Implement pre-commit hooks for secret detection

4.3 Automate Security and Compliance Scanning

Microsoft Defender for Cloud DevOps Security

- **Enablement**: Connect GitHub org or Azure DevOps project in Defender portal, install GitHub App
- **Capabilities**: Secret scanning, code scanning (CodeQL), dependency scanning, Infrastructure as Code scanning
- **Alerts**: Integrated into Security tab, assign severity, track remediation
- **Compliance**: Map to PCI-DSS, SOC2, ISO 27001 frameworks
- **Documentation**: https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-devops-introduction

GitHub Advanced Security

GitHub Actions - CodeQL Analysis:

```
- name: Initialize CodeQL
  uses: github/codeql-action/init@v2
  with:
    languages: csharp, javascript

- name: Perform CodeQL Analysis
  uses: github/codeql-action/analyze@v2

# Enable in repository settings
# Security > Code scanning alerts
```

GitHub Advanced Security Integration:

```
steps:
  - task: AdvancedSecurity-Codeql-Init@1
    inputs:
      languages: 'csharp,javascript'
  - task: AdvancedSecurity-Dependency-Scanning@1
  - task: AdvancedSecurity-Codeql-Analyze@1
```

- **CodeQL**: Create custom queries, schedule scans, analyze third-party dependencies
- **Secret Scanning**: Enable for private repos, receive alerts, revoke secrets via API
- **Dependabot**: Configure .github/dependabot.yml, daily scans, auto-merge security updates, group updates

Container Scanning

- **Trivy**: Scan for OS packages and application dependencies, integrate in pipeline
- **Microsoft Defender**: Scan Azure Container Registry images, block vulnerable images
- **GitHub Container Scanning**: Use CodeQL in container, scan base images
- **Scan container images** with Trivy or Anchore
- **Implement image signing** with Notation
- **Scan for CVE vulnerabilities**

Open Source Compliance

- FOSSA/WhiteSource: License compliance, attribution reports, policy enforcement
- GitHub Dependency Graph: View dependencies, vulnerable versions, license information
- SBOM Generation: Generate Software Bill of Materials, attach to releases, compliance requirements

Infrastructure Security

- Use Checkov for IaC scanning
- Implement Azure Policy for resource compliance
- Scan ARM/Bicep templates with PSRule

Compliance Automation

- Configure Component Governance for open-source compliance
- Automate license scanning with Mend Bolt
- Generate Software Bill of Materials (SBOM)
- Implement audit trails for all deployments

OWASP ZAP Integration

- Run OWASP ZAP for web app security scanning
- Integrate into release pipeline gates
- Fail pipeline on high-severity findings

---

5. IMPLEMENT AN INSTRUMENTATION STRATEGY (5-10%)

5.1 Configure Monitoring for DevOps Environment

Azure Monitor Integration

- **Log Analytics Workspace**: Centralized logging, data retention 30-730 days, data export to Storage/Event Hub
- **Azure Monitor Agent**: Install on VMs, automatic extension on Azure VMs, data collection rules
- **DevOps Tool Integration**: Azure Pipelines logging to Log Analytics, GitHub Actions with Azure Monitor logs
- **Application Insights** for application performance

- **Log Analytics** for centralized logging
- **Azure Monitor alerts** for pipeline events
- **Prometheus managed service** for metrics

Pipeline Monitoring

- Enable pipeline analytics
- Configure alerts for:
  - Pipeline failures
  - Deployment duration exceeding threshold
  - Test failure rates
  - Agent pool capacity issues

Environment Monitoring

- Monitor deployed application health
- Track resource utilization and costs
- Set up availability tests
- Configure service health alerts

Application Insights Configuration

Enable Application Insights for App Service:

```
- task: AzureAppServiceSettings@1
  inputs:
    azureSubscription: 'AzureServiceConnection'
    appName: 'my-app'
    resourceGroupName: 'my-rg'
    appSettings: |
      {
        "name": "APPINSIGHTS_INSTRUMENTATIONKEY",
        "value": "$(appInsightsKey)",
        "slotSetting": false
      }
```

GitHub Actions - Update Application Settings:

```
- uses: azure/appservice-settings@v1
  with:
    app-name: 'my-app'
    app-settings.json: |
      {
        "name": "APPINSIGHTS_INSTRUMENTATIONKEY",
        "value": "${{ secrets.APP_INSIGHTS_KEY }}"
      }
```

Application Insights Configuration (Alternative):

```
variables:
  applicationInsightsKey: $(appInsightsKey)

steps:
  - task: AzureAppServiceSettings@1
    inputs:
      azureSubscription: 'MySubscription'
      appName: 'my-app'
      appSettings: |
        [
          {
            "name": "APPINSIGHTS_INSTRUMENTATIONKEY",
            "value": "${applicationInsightsKey}"
          }
        ]
```

Telemetry Collection

- **Application Insights**: Auto-instrumentation for .NET, Java, Node.js, manual SDK for custom events, dependency tracking, distributed tracing
- **VM Insights**: Performance counters, process information, map dependencies
- **Container Insights**: AKS cluster monitoring, container logs, Prometheus metrics, recommend using Azure Monitor managed service for Prometheus
- **Storage Insights**: Transaction metrics, capacity trends, diagnostic logging
- **Network Insights**: NSG flow logs, Traffic Analytics, connection monitoring

GitHub Monitoring

- **Insights Tab**: View actions usage, runner minutes, workflow success rates, repository activity
- **Audit Logs**: Organization-level events, export to Splunk/Azure Sentinel, SIEM integration
- **API Rate Limits**: Monitor via HTTP headers, implement backoff strategies

Alert Configuration

- **Azure Monitor Alerts**: Metric alerts (thresholds), Log alerts (KQL queries), Activity log alerts, Smart Detection (AI-powered)
- **Action Groups**: Email, SMS, push notifications, Logic App/Webhook, Azure Function, ITSM integration
- **GitHub Actions Alerts**: Failed workflow notifications, dependabot alerts, secret scanning alerts

5.2 Analyze Metrics from Instrumentation

Infrastructure Performance Indicators

- **CPU/Memory**: Top 10 processes by CPU, available memory, page faults, context switches
- **Disk**: Queue length, read/write latency, free space percentage, IOPS

- **Network**: Bytes sent/received, TCP connections, retransmits, latency

KQL Queries for Azure Monitor

Application Insights - Failed requests:

```
requests
| where success == false
| summarize Count=count() by name, resultCode
| order by Count desc
```

VM Insights - High CPU:

```
InsightsMetrics
| where Origin == "vm.azm.ms"
| where Namespace == "Processor"
| where Name == "UtilizationPercentage"
| summarize AvgCPU=avg(Val) by Computer
| where AvgCPU > 80
```

Container Insights - Restarted containers:

```
KubePodInventory
| where ContainerRestartCount > 5
| project Namespace, Name, ContainerRestartCount
```

Pipeline failure analysis:

```
AzureDevOpsActivityLogs
| where OperationName == "PipelineRun" and Result == "Failed"
| summarize count() by PipelineName, bin(TimeGenerated, 1d)
```

Distributed Tracing

- **Application Map**: Visualize dependencies between components, identify bottlenecks, failure rates
- **End-to-End Transactions**: Track requests across microservices, correlate logs, view performance bottlenecks
- **Performance/Failures Blades**: Drilling into specific operations, analyze trends, set up alerts
- **Custom Telemetry**: Track business metrics, feature usage, user journeys

Custom Metrics

- Track business metrics via Application Insights
- Create custom dashboards with KQL queries
- Implement distributed tracing across services
- Use Azure Monitor Workbooks for analysis

Feedback Mechanisms

- Integrate user feedback with Azure DevOps work items
- Implement feature flags for gradual rollout
- Use Application Insights user feedback
- Create automated reports for stakeholders

Log Analysis Best Practices

- **Structured Logging**: Use JSON format, include correlation IDs, log levels (Trace, Debug, Info, Warn, Error, Critical)
- **Sampling**: Adaptive sampling in Application Insights, configure excluded types, preserve full fidelity for errors
- **Retention**: Application Insights data retention 90 days (standard), 2 years (with Log Analytics), continuous export to Storage for long-term
- **Cost Optimization**: Daily cap, sampling adjustments, filter unnecessary telemetry, use Basic Logs tier for verbose logs

---

KEY TOOLS AND SERVICES REFERENCE

Tool/Service	Primary Use	Key Features
Azure Pipelines	CI/CD automation	YAML/Classic pipelines, multi-stage deployments
Azure Repos	Source control	Git repositories, pull requests, branch policies
Azure Artifacts	Package management	NuGet, npm, Maven feeds, upstream sources
Azure Boards	Work tracking	Kanban boards, backlogs, sprint planning
GitHub Advanced Security	Security scanning	CodeQL, secret scanning, dependency scanning
Azure Key Vault	Secrets management	Keys, secrets, certificates with RBAC
Azure Monitor	Monitoring & logging	Application Insights, Log Analytics, alerts
Bicep	Infrastructure as code	Declarative Azure resource deployment
Azure Policy	Compliance	Resource governance, compliance enforcement
Microsoft Defender for DevOps	DevOps security	Vulnerability management, compliance scanning

---

BEST PRACTICES SUMMARY

Security Best Practices

- Implement Zero Trust principles: never trust, always verify
- Use managed identities instead of secrets
- Enable MFA for all users
- Regularly rotate secrets and certificates
- Scope service connections to specific resources
- Enable auditing and review logs regularly

Pipeline Best Practices

- Use YAML pipelines over Classic for version control
- Implement infrastructure as code for all resources
- Break pipelines into reusable templates
- Use parallel jobs for faster execution
- Implement approval gates for production
- Cache dependencies to reduce build time
- Keep pipelines simple and focused

Source Control Best Practices

- Use feature branches for all development
- Require pull requests with code review
- Implement branch policies with build validation
- Keep main branch always deployable
- Use semantic versioning for releases
- Write meaningful commit messages
- Link commits to work items

Monitoring Best Practices

- Implement observability across all layers
- Set up proactive alerts before issues occur
- Use distributed tracing for microservices
- Monitor pipeline health and performance
- Track deployment metrics and DORA metrics
- Create actionable dashboards for teams

EXAM PREPARATION CHECKLIST

Prerequisites Verification

- ☑ Hold current AZ-104 OR AZ-204 certification
- ☑ 2+ years hands-on Azure experience (administration or development)
- ☑ Strong Git knowledge (branching, merging, rebasing)
- ☑ CI/CD fundamentals understanding

Hands-On Labs Requirements

- **Azure Subscription**: Active pay-as-you-go or Visual Studio subscription
- **GitHub Account**: Personal or organization account with GitHub Advanced Security trial
- **Azure DevOps Organization**: Create free organization at dev.azure.com
- **Tools**: Install Azure CLI, Git, VS Code with Azure extensions

Key Practice Areas

1. Build 3-5 complete pipelines combining CI/CD, security scanning, and infrastructure deployment
2. Configure cross-platform integrations between GitHub, Azure DevOps, and Azure services
3. Implement full monitoring stack with Application Insights, Log Analytics, and custom dashboards
4. Set up security controls using Managed Identities, Key Vault, and Defender for Cloud
5. Practice KQL queries on sample Log Analytics data

Microsoft Learn Paths

- AZ-400: Get started on a DevOps transformation journey (4.5 hours)
- AZ-400: Development for enterprise DevOps (6 hours)
- AZ-400: Implement CI with Azure Pipelines and GitHub Actions (5 hours)
- AZ-400: Design and implement a release strategy (4 hours)
- AZ-400: Implement a secure continuous deployment (3 hours)
- AZ-400: Manage infrastructure as code (3 hours)

Documentation Deep Dives

- **Azure DevOps**: Review all Pipelines documentation, especially YAML syntax, templates, and expressions
- **GitHub Actions**: Study workflow syntax, reusable workflows, and security hardening
- **Azure Monitor**: Master Application Insights SDK configuration and KQL query language
- **Azure Security**: Understand Defender for Cloud integration and compliance frameworks

---

COMMON EXAM PITFALLS

1. **YAML Syntax Errors**: Misaligned indentation, incorrect task names, missing required fields

2. **Authentication Confusion**: Mixing up Service Principals vs Managed Identities, incorrect scope assignments
3. **Deployment Strategy Misapplication**: Choosing wrong strategy for given RTO/RPO requirements
4. **Metrics Overload**: Not knowing which metrics apply to which DevOps phase
5. **Security Scanning Configuration**: Forgetting to enable Defender for Cloud or GitHub Advanced Security features
6. **KQL Query Structure**: Incorrect table names, missing time filters, improper aggregation functions

---

FINAL EXAM DAY TIPS

- **Time Management**: 50-60 questions, ~3 minutes per question, flag complex scenario questions for review
- **Scenario Focus**: Most questions are scenario-based; identify key constraints (cost, security, compliance)
- **Hands-On Labs**: Expect 1-2 performance-based labs requiring pipeline configuration or KQL queries
- **Documentation Access**: No external resources allowed; memorize key YAML syntax and Azure service capabilities
- **Question Types**: Multiple choice, drag-and-drop (ordering steps), case studies with multiple questions
- **Hands-on practice is critical**: Set up your own Azure DevOps environment
- **Understand the "why"**: Know the reasoning behind each practice, not just the "how"
- **Focus on high-weightage areas**: Spend 50% of study time on pipelines section
- **Review official documentation**: Use Microsoft Learn labs and modules
- **Practice tests**: Take practice exams to understand question format
- **Case studies**: Be prepared for scenario-based questions
- **YAML syntax**: Memorize common pipeline syntax patterns
- **Security first**: Always consider security implications in scenarios

---

Sources: All content derived from official Microsoft Learn documentation including AZ-400 Study Guide, Azure DevOps documentation, and Exam Readiness Zone.

Good luck with your AZ-400 certification journey!